

Nama : Achmad Pradita Dwi Firmansyah
NIM : 254107020130
Kelas / Absen : TI-1G.01

JOB SHEET 14

Tree

Implementasi Binary Search Tree menggunakan Linked List (100 Menit)

14.1.1 Kode Program

Class Mahasiswa01

```
package Pertemuan14;
```

```
public class Mahasiswa01 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    Mahasiswa01() {
```

```
    }
```

```
    Mahasiswa01(String nim, String nama, String kelas, double ipk) {
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.kelas = kelas;
```

```
        this.ipk = ipk;
```

```
    }
```

```
    void tampilInformasi() {
```

```
        System.out.println("NIM: " + this.nim + " " + "Nama: " + this.nama + " " +  
        "Kelas: " + this.kelas + " "
```

```
        + "IPK: " + this.ipk);
```

```
    }  
}
```

Class Node01

```
package Pertemuan14;
```

```
public class Node01 {
```

```
    Mahasiswa01 mahasiswa;
```

```
    Node01 left, right;
```

```
    Node01() {
```

```
    }
```

```
    Node01(Mahasiswa01 mahasiswa) {
```

```
        this.mahasiswa = mahasiswa;
```

```
        left = right = null;
```

```
    }
```

```
}
```

Class BinaryTree01

```
package Pertemuan14;
```

```
public class BinaryTree01 {
```

```
    Node01 root;
```

```
    BinaryTree01() {
```

```
        root = null;
```

```
    }
```

```
    boolean isEmpty() {
```

```
        return root == null;
```

```
    }
```

```

void add(Mahasiswa01 mahasiswa) {
    Node01 newNode = new Node01(mahasiswa);
    if (isEmpty()) {
        root = newNode;
    } else {
        Node01 current = root;
        Node01 parent = null;
        while (true) {
            parent = current;
            if (mahasiswa.ipk < current.mahasiswa.ipk) {
                current = current.left;
                if (current == null) {
                    parent.left = newNode;
                    return;
                }
            } else {
                current = current.right;
                if (current == null) {
                    parent.right = newNode;
                    return;
                }
            }
        }
    }
}

```

```

boolean find(double ipk) {
    boolean result = false;
    Node01 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {

```

```

        current = current.right;
    } else {
        current = current.left;
    }
}
return result;
}

```

```

void traversePreOrder(Node01 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

```

```

void traverseInOrder(Node01 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

```

```

void traversePostOrder(Node01 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

Node01 getSuccessor(Node01 del) {
    Node01 successor = del.right;
    Node01 successorParent = del;

```

```

while (successor.left != null) {
    successorParent = successor;
    successor = successor.left;
}
if (successor != del.right) {
    successorParent.left = successor.right;
    successor.right = del.right;
}
return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node01 parent = root;
    Node01 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // penghapusan

```

```

if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    // jika tidak ada anak (leaf), maka node dihapus
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    }
    } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    }
    } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            } else {
                parent.right = current.left;
            }
        }
    }
}

```

```

    } else { // jika punya 2 anak
        Node01 successor = getSuccessor(current);
        /*
        * System.out.println("Jika 2 anak, current = ");
        * successor.mahasiswa.tampilInformasi();
        */
        if (current == root) {
            root = successor;
        } else {
            if (isLeftChild) {
                parent.left = successor;
            } else {
                parent.right = successor;
            }
        }
        successor.left = current.left;
    }
}
}
}
}

```

Class BinaryTreeMain01

```
package Pertemuan14;
```

```
public class BinaryTreeMain01 {
```

```
    public static void main(String[] args) {
```

```
        BinaryTree01 bst = new BinaryTree01();
```

```
        bst.add(new Mahasiswa01("244160121", "Ali", "A", 3.57));
```

```
        bst.add(new Mahasiswa01("244160221", "Badar", "B", 3.85));
```

```
        bst.add(new Mahasiswa01("244160185", "Candra", "C", 3.21));
```

```
        bst.add(new Mahasiswa01("244160220", "Dewi", "B", 3.54));
```

```
        System.out.println("\nDaftar semua mahasiswa (in order traversal) :");
```

```

        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa01("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa01("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa01("244160170", "Fizi", "B", 3.46));
        System.out.println("\nDaftar semua mahasiwa setelah penambahan 3
mahasiswa:");
        System.out.println("InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("\nPreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

14.1.2 Hasil Run Terminal


```
Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

```
Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan
```

```
Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

```
PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
```

```
PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
```

Penghapusan data mahasiswa

```
Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

```
Daftar semua mahasiswa (in order traversal) :
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

```
Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan
```

```
Daftar semua mahasiwa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
```

```
PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
```

```

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

Penghapusan data mahasiswa

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>

```

14.1.3 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Karena pada binary tree nilai yang lebih kecil dari nilai sekarang berada pada kiri dan nilai yang lebih besar dari nilai sekarang ada di sebelah kanan, jadi proses pencarian bisa lebih mudah karena hanya menggunakan salah satu bagian

2. Untuk apakah di class **Node**, kegunaan dari atribut **left** dan **right**?
Kegunaan atribut left dan right sebagai pointer pada node untuk menunjuk ke node berikutnya, left menunjuk ke node sebelah kiri dimana node sebelah kiri akan lebih kecil, dan right akan menunjuk ke node sebelah kanan yang node nya lebih besar
3. a. Untuk apakah kegunaan dari atribut **root** di dalam class **BinaryTree**?
Atribut root adalah atribut awal atau akar dari tree dimana semua akan dimulai dari root

b. Ketika objek tree pertama kali dibuat, apakah nilai dari **root**?
Ketika objek tree pertama kali dibuat, nilai dari root adalah null karena belum ada data yang ditambahkan
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?
Ketika tree masih kosong, dan akan ditambahkan node baru, akan dilakukan proses pengecekan tree masih kosong, jika root == null, maka node baru akan di tambahkan ke root
5. Perhatikan method **add()**, di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```

parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}

```

Program pertama menginisialisasikan parent dengan current, kemudian jika node baru nilai ipk nya kurang dari parent, maka node baru akan diarahkan ke sebelah kiri, kemudian dilakukan pengecekan jika node sebelah kiri kosong maka node baru akan di tambahkan di sana dan program akan keluar. Namun jika node baru nilai ipk lebih dari parent, maka node baru akan ditambahkan di sebelah kanan, kemudian dilakukan pengecekan jika node sebelah kanan kosong, maka node baru akan ditambahkan di sana lalu program akan keluar.

6. Jelaskan langkah-langkah pada method **delete()** saat menghapus sebuah node yang memiliki dua anak. Bagaimana method **getSuccessor()** membantu dalam proses ini?

Pertama akan di lakukan pencarian node yang berada dibawah node yang akan dihapus, kemudian jika node yang akan dihapus adalah root, maka successor akan menggantikan root, jika node yang dihapus bukan root, maka akan dilakukan pengecekan jika leftchild, maka successor akan dijadikan parent kiri, namun jika tidak akan dijadikan parent kanan. Kemudian akan di sambungkan successor ke bagian kiri sama dengan node yang dihapus yang mengarah ke leftchild.

14.2 Kegiatan Praktikum 2

Implementasi Binary Tree dengan Array (45 Menit)

14.2.1 Kode Program

Class BinaryTreeArray01

```
package Pertemuan14;

public class BinaryTreeArray01 {
    Mahasiswa01[] dataMahasiswa;
    int idxLast;

    BinaryTreeArray01() {
        this.dataMahasiswa = new Mahasiswa01[10];
    }

    void populateData(Mahasiswa01 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2 * idxStart + 1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

Class BinaryTreeArrayMain01

```
package Pertemuan14;

public class BinaryTreeArrayMain01 {
```

```

public static void main(String[] args) {
    BinaryTreeArray01 bta = new BinaryTreeArray01();
    Mahasiswa01 mhs1 = new Mahasiswa01("244160121", "Ali", "A", 3.57);
    Mahasiswa01 mhs2 = new Mahasiswa01("244160185", "Candra", "C", 3.41);
    Mahasiswa01 mhs3 = new Mahasiswa01("244160221", "Badar", "B", 3.75);
    Mahasiswa01 mhs4 = new Mahasiswa01("244160220", "Dewi", "B", 3.35);

    Mahasiswa01 mhs5 = new Mahasiswa01("244160131", "Devi", "A", 3.48);
    Mahasiswa01 mhs6 = new Mahasiswa01("244160205", "Ehsan", "D", 3.61);
    Mahasiswa01 mhs7 = new Mahasiswa01("244160170", "Fizi", "B", 3.86);

    Mahasiswa01[] dataMahasiswa = { mhs1, mhs2, mhs3, mhs4, mhs5, mhs6,
    mhs7, null, null, null };
    int idxLast = 6;
    bta.populateData(dataMahasiswa, idxLast);
    System.out.println("\nInOrder Traversal Mahasiswa: ");
    bta.traverseInOrder(0);
}
}

```

14.2.2 Hasil Run Terminal

```

Inorder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86

InOrder Traversal Mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>

```

14.2.3 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class **BinaryTreeArray**?

Atribut data merupakan array yang menyimpan object mahasiswa, dan idxLast adalah atribut yang menyimpan nilai indeks data yang terakhir di tambahkan

2. Apakah kegunaan dari method **populateData()**?
Method populateData() digunakan untuk menginisialikan banyak data mahasiswa ke array tree, dan menentukan indeks terakhir array yang ditambahkan
3. Apakah kegunaan dari method **traverseInOrder()**?
Method traverseInOrder() digunakan untuk menampilkan seluruh data dari leftchild, parent, rightchild, sehingga akan menampilkan data yang urut dari ipk terkecil ke terbesar
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
Posisi left child pada indeks $2*2+1 = 5$, dan right child pada indeks $2*2+2 = 6$
5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?
Kegunaan `int idxLast = 6` adalah untuk menginisialisasikan data mahasiswa terakhir yang sudah di tambah sampai di indeks ke 6
6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?
Karena indeks $2*idxStart+1$ dan $2*idxStart+2$ merupakan rumus untuk menentukan indeks leftchild dan rightchild pada struktur pohon biner yang terdiri dari 2 subtree yaitu kiri dan kanan.